DATABASE SYSTEMS PROJECT

UniSphere ERP — University Management System

Project Proposal

Project Team	
Abdullah Hossam	24030072
Nada Ibrahim	24030046
Rewan Tamer	24030140
Menna Ahmed	24030183

Faculty of Computers & Information Technology

Academic Year: 2025 / 2026

May 2026

1. Project Overview

UniSphere ERP is a full-stack University Management System built to manage all core academic and administrative operations of a university. The system integrates a fully relational SQL Server database with a modern React-based web application, enabling role-based access for administrators, faculty, and students.

The project delivers an end-to-end solution covering database design, a RESTful backend API, and an interactive single-page application — all working together to digitize every aspect of university operations.

1.1 Project Objectives

- Design and implement a normalized relational database for a university management domain.
- Build a full-stack web application (frontend + backend API) consuming the database.
- Support complete CRUD operations across all entities.
- Implement role-based authentication (Admin, Employee, Student).
- Generate analytical reports and statistical dashboards.

1.2 Technologies Used

Layer	Technology	Purpose
Database	Microsoft SQL Server	Core data storage & queries
ORM / Schema	Prisma (SQLite for dev)	Schema definition & migrations
Backend	Node.js + Express + TypeScript	REST API server
Frontend	React 19 + TypeScript + Vite	Single-page application
Styling	Tailwind CSS v4	UI design & layout
Auth	JWT + bcryptjs	Secure role-based login
Charts	Recharts	Dashboard analytics
Animations	Motion/React (Framer)	UI transitions

2. Database Design

The database was designed following the relational model with full normalization up to the Third Normal Form (3NF). It consists of 12 interrelated tables covering every aspect of university management.

2.1 Entity-Relationship Summary

The schema resolves a circular dependency between Departments and Employees (DeanID) using a deferred foreign key added via ALTER TABLE after both tables are created. All primary keys use IDENTITY(1,1) auto-increment.

2.2 Tables & Schema

#	Table	Primary Key	Key Relationships
1	Departments	DepartmentID	DeanID → Employees (circular, deferred)
2	Semesters	SemesterID	Referenced by Schedule, Fees
3	Rooms	RoomID	Referenced by Schedule, Exams
4	Employees	EmployeeID	DepartmentID → Departments
5	Students	StudentID	DepartmentID → Departments
6	Courses	CourseID	DepartmentID → Departments
7	Schedule	ScheduleID	CourseID, EmployeeID, RoomID, SemesterID
8	Enrollments	EnrollmentID	StudentID, ScheduleID; UNIQUE(Student, Schedule)
9	Exams	ExamID	ScheduleID, RoomID
10	Fees	FeeID	StudentID, SemesterID
11	Scholarships	ScholarshipID	StudentID
12	Attendance	AttendanceID	EnrollmentID; UNIQUE(Enrollment, Date)

2.3 Constraints & Data Integrity

The schema enforces strong data integrity through CHECK constraints on every table:

- Rooms.RoomType: must be in { Lecture, Lab, Section, Electronics Lab }
- Employees.StaffType: must be in { Academic, Non-Academic }
- Students.GPA: range 0.00 – 4.00
- Courses.CreditHours: must be > 0
- Schedule.Day_Of_Week: valid weekday names only; EndTime > StartTime
- Enrollments.Grade: 0 – 100 or NULL; Status in { Active, Dropped, Completed }
- Fees.PaidAmount: 0 ≤ PaidAmount ≤ Amount; Status in { Paid, Partial, Unpaid }
- Scholarships & Semesters: EndDate > StartDate
- Attendance.Status: in { Present, Absent, Late }

- UNIQUE constraints prevent duplicate enrollment (Student + Schedule) and duplicate attendance record (Enrollment + Date).

2.4 Sample Data

Entity	Count Inserted	Notes
Departments	6	CS, Engineering, Physical Therapy, Business, Art, Nurse
Semesters	2	Fall 2025, Spring 2026
Rooms	10+	Lecture, Lab, Section, Electronics Lab types
Employees	9+	Academic & Non-Academic; 2 assigned as deans
Students	10+	Spread across departments
Courses	11+	Linked to departments
Schedule	11	Cross-referenced with rooms, employees, semesters
Enrollments	15+	With grades and status
Exams	22	Midterm + Final for each schedule
Fees	~12	Paid, Partial, Unpaid statuses
Scholarships	~8	Various amounts and durations
Attendance	30+	Present, Absent, Late statuses

3. SQL Queries

17 analytical queries were written to extract meaningful information from the database, ranging from simple selects to multi-join and correlated subquery patterns.

No.	Query Name	Description & Technique
Q1	Students ordered by GPA	Simple SELECT + ORDER BY on Students table to rank students by academic performance.
Q2	Students with department names	INNER JOIN between Students and Departments; retrieves DepartmentName alongside student names.
Q3	Students with unpaid fees	JOIN Students and Fees with WHERE Status = 'Unpaid'; identifies students with outstanding balances.
Q4	Employees with department names	JOIN Employees and Departments to display full staff roster with their organizational unit.
Q5	Employees and courses they teach	JOIN Employees and Courses via matching DepartmentID; maps instructors to department course catalog.
Q6	Average GPA per department	GROUP BY DepartmentName with AVG(GPA); ranks departments by average student academic performance.
Q7	Full schedule details	4-way JOIN: Schedule, Courses, Employees, Rooms, Semesters — gives complete timetable view.
Q8	Students enrolled per course + grade	JOIN Enrollments, Students, Schedule, Courses; shows grade distribution per course.
Q9	Attendance summary per student	Conditional aggregation (SUM + CASE WHEN) counts Present, Absent, Late per student.
Q10	Exam schedule with room & course	JOIN Exams, Schedule, Courses, Rooms, Semesters; full exam timetable ordered by date.
Q11	Students with scholarships	JOIN Scholarships and Students; lists scholarship recipients with amount and validity period.
Q12	Fee payment status + remaining balance	Computes (Amount - PaidAmount) AS RemainingAmount; useful for finance tracking.
Q13	Number of students per department	LEFT JOIN + COUNT + GROUP BY; includes departments with zero students.
Q14	Instructors and courses assigned	LEFT JOIN + COUNT on Schedule filtered by StaffType = 'Academic'; shows teaching load.
Q15	Students with GPA above dept average	Correlated subquery: compares each student's GPA to their own department's average.
Q16	Total salary cost per staff type	GROUP BY StaffType with COUNT, SUM, AVG; payroll breakdown by employment category.

Q17	Room utilization	LEFT JOIN Rooms and Schedule + COUNT; identifies busiest and idle rooms.

3.1 Query Technique Coverage

- Basic SELECT / WHERE / ORDER BY — Q1, Q3
- INNER JOIN (2–5 tables) — Q2, Q4, Q5, Q7, Q8, Q10, Q11, Q12
- LEFT JOIN (including zero-count rows) — Q13, Q14, Q17
- GROUP BY + Aggregate functions (COUNT, AVG, SUM) — Q6, Q9, Q13, Q14, Q16, Q17
- Conditional aggregation (CASE WHEN inside SUM) — Q9
- Correlated subquery — Q15
- Computed columns (Amount - PaidAmount) — Q12

4. Database Views

Three reusable views were created to encapsulate frequently-used query logic and simplify application-level data access.

View	Description
vw_StudentProfile	Provides a complete academic snapshot for each student: full name, email, department name, GPA, and enrollment date. Joins Students with Departments and is used wherever a combined student profile is needed without repeating the join logic.
vw_ActiveEnrollments	Shows all currently active course enrollments enriched with student name, course name, semester, and current grade. Filters WHERE Status = 'Active' so only ongoing registrations are returned. Used by the scheduling and academic progress modules.
vw_OutstandingFees	Lists all students with Partial or Unpaid fee records, showing semester name, original amount, amount paid, outstanding balance (computed column), due date, and status. Directly feeds the Finance module's outstanding balances dashboard.

5. Web Application — UniSphere ERP

The database was integrated into a fully functional full-stack web application. The application exposes all database tables through a RESTful API and renders an interactive management interface in the browser.

5.1 Architecture

A single Node.js/Express server handles both the REST API and serves the compiled React frontend via Vite. Prisma ORM provides a type-safe layer above the SQLite database, mirroring the SQL Server schema exactly.

Component	Description
server.ts	Main Express server (~740 lines). Registers all REST API routes, handles JWT auth middleware, and serves the Vite-built frontend in production.
prisma/schema.prisma	Database schema definition with 12 Prisma models matching the SQL Server schema. Also used for migrations and type generation.
src/App.tsx	Root React component — manages authentication state (JWT), renders Sidebar + Header layout, and defines all client-side routes.
src/pages/	10 page components: Dashboard, Students, Employees, Departments, Courses, Schedules, Attendance, Finance, Scholarships, Reports.
src/components/	Shared UI components: Sidebar (navigation), Header (user info), Modal (create/edit dialogs).
prisma/seed.ts	Database seeder — inserts representative test data across all 12 tables.

5.2 REST API Endpoints

The server exposes 35+ REST endpoints covering full CRUD for all entities:

Method	Endpoint	Description
POST	/api/auth/login	Authenticate user, return JWT token + role
GET	/api/departments	List all departments with dean and counts
POST	/api/departments	Create a new department
PUT	/api/departments/:id	Update department details
DELETE	/api/departments/:id	Remove a department
GET	/api/students	List students with department info
POST	/api/students	Enroll a new student
PUT	/api/students/:id	Update student record
DELETE	/api/students/:id	Remove student + cascade user
GET	/api/employees	List employees with department info

POST	/api/employees	Add new employee
PUT	/api/employees/:id	Update employee record
DELETE	/api/employees/:id	Remove employee
GET	/api/courses	List all courses
POST	/api/courses	Add a new course
PUT	/api/courses/:id	Update course details
DELETE	/api/courses/:id	Remove a course
GET	/api/schedules	Full schedule with joins
POST	/api/schedules	Create a schedule slot
DELETE	/api/schedules/:id	Remove a schedule slot
GET	/api/attendance	Attendance records (filterable)
POST	/api/attendance	Record attendance
GET	/api/fees	All fee records
POST	/api/fees	Create fee record
PUT	/api/fees/:id	Update payment status
DELETE	/api/fees/:id	Remove fee record
GET	/api/scholarships	All scholarships
POST	/api/scholarships	Assign scholarship
PUT	/api/scholarships/:id	Update scholarship
DELETE	/api/scholarships/:id	Remove scholarship
GET	/api/stats	Dashboard statistics aggregate
GET	/api/rooms	List all rooms
GET	/api/semesters	List all semesters
POST	/api/semesters	Create semester
GET	/api/enrollments	Student course enrollments
POST	/api/enrollments	Enroll student in course

5.3 Authentication & Security

- JWT (JSON Web Tokens) with 24-hour expiry.
- Passwords hashed with bcryptjs (salt rounds = 10).
- Three roles: ADMIN, EMPLOYEE, STUDENT.
- Auto-registration for @iu.edu.eg domain emails with ADMIN role.
- Protected routes on the frontend using localStorage token persistence.

5.4 Application Pages

Page	Features
Dashboard	KPI cards (Total Students, Active Courses, Revenue, Departments); Area chart for revenue vs expenses; Pie chart for course distribution by faculty; Animated stat cards with trend indicators.
Students	Full student list with search/filter; Add, Edit, Delete student records; GPA display; Department assignment; Fee status indicator.
Employees	Staff directory with job titles and staff type (Academic / Non-Academic); Salary display; Department link; Add/Edit/Delete operations.
Departments	Department cards with dean name, student count, employee count; Assign/change dean; CRUD operations.
Courses	Course catalog with credit hours; Department filter; Full CRUD.
Schedules	Weekly timetable view; Assign course, instructor, room, semester, day, time slot; Conflict-aware display.
Attendance	Record daily attendance per student per enrollment; Filter by date, course, student; Present/Absent/Late status.
Finance	Fee management per student per semester; Payment tracking (Paid/Partial/Unpaid); Balance computation; Add/Edit/Delete payments.
Scholarships	Scholarship registry per student; Amount, start/end date; CRUD operations.
Reports	Statistical summaries and exportable report views across academic and financial data.

6. System Flow & Data Relationships

6.1 Authentication Flow

- User submits email + password to POST /api/auth/login.
- Server checks Prisma Users table and validates password with bcrypt.
- On success: JWT signed with user ID, email, and role — returned to client.
- Client stores token in localStorage and includes it in subsequent API calls.
- Protected pages redirect to Login if no valid token is present.

6.2 Student Lifecycle

- Student registered → record created in Students table, linked to a Department.
- Student enrolled in courses → Enrollment records created (Schedule-linked).
- Attendance tracked per session → Attendance records (Enrollment-linked).
- Grades assigned at end of semester → Enrollment.Grade updated.
- Fees assessed per semester → Fee records created; payments tracked.
- Scholarships applied → Scholarship records linked to student.

6.3 Academic Management Flow

- Department created → assigned a Dean (Employee with Academic StaffType).
- Courses added → linked to Departments and assigned credit hours.
- Semester defined → named with start/end date.
- Schedule created → links Course + Employee (Instructor) + Room + Semester + Day/Time.
- Exams scheduled → linked to Schedule and Room with type (Midterm/Final).

7. Challenges & Solutions

Challenge	Solution Applied
Circular FK: Departments ↔ Employees (DeanID)	Created Departments without FK, inserted Employees first, then added FK via ALTER TABLE after both tables exist.
Preventing duplicate enrollments	Added UNIQUE constraint on (StudentID, ScheduleID) in Enrollments table.
Preventing duplicate attendance records	Added UNIQUE constraint on (EnrollmentID, Date) in Attendance table.
Grade validation allowing NULL for pending	CHECK constraint: Grade IS NULL OR (Grade >= 0 AND Grade <= 100).
Fee payment amount validation	CHECK: PaidAmount >= 0 AND PaidAmount <= Amount to prevent over-payment.
Room type restrictions	CHECK constraint with IN clause limits RoomType to the 4 defined values.
Full-stack data consistency	Prisma schema mirrors SQL Server design; cascade deletes ensure orphan records are cleaned up.
Dashboard real-time stats	Dedicated /api/stats endpoint aggregates counts and totals server-side to avoid N+1 queries.

8. Team Contributions

This project was completed by a team of four. All implementation work, database design, query writing, and web development were performed by the team members.

Name	Student ID	Contributions
Abdullah Hossam	24030072	Database schema design (DDL), constraint implementation, SQL queries (Q1–Q9), team coordination and integration.
Nada Ibrahim	24030046	SQL queries (Q10–Q17), Views (vw_StudentProfile, vw_ActiveEnrollments), sample data insertion and verification.
Rewan Tamer	24030140	Backend REST API development (Express server, all GET endpoints), Prisma schema, authentication system.
Menna Ahmed	24030183	Frontend React application (all 10 pages), UI/UX design, Dashboard charts, Finance and Scholarships modules.

9. Conclusion

The UniSphere ERP project successfully demonstrates the end-to-end implementation of a university management system spanning database design, backend API development, and a modern web frontend.

Key achievements of this project include:

- A fully normalized 12-table SQL Server relational database with comprehensive integrity constraints.
- 17 SQL queries covering a wide range of analytical and operational use cases.
- 3 reusable views encapsulating complex joins for the most common data access patterns.
- A complete REST API with 35+ endpoints supporting all CRUD operations.
- A responsive, role-based React web application with 10 functional modules.
- Secure JWT-based authentication with three user roles.

The system is production-ready in architecture and design, and can be extended with additional features such as notification systems, grade report PDF export, and a student portal self-service module.

10. Project Deliverable Links

All project files and diagrams are publicly accessible via the links below:

Deliverable	
GitHub Repository	github.com/eng00abdullah/University-Database
ERD Diagram	View ERD on diagrams.net
Schema Diagram	View Schema on diagrams.net
SQL Code File	University_Management_Database_System.sql — Google Drive

All files are also available inside the GitHub repository project folder.